

C04 ASSESSMENT TOOL 1-CASE STUDY

Name: Y.Thanushma

Registration Number:192425355

Course Code: CSA0617(Design & Analysis of Algoorithms)

1.Digital Examination Result Search System Using Binary Search

Introduction

A university maintains examination results of students in a database. Each student record contains information such as registration number, student name, marks, grade, and result status. Since registration numbers are unique and can be stored in sorted order, Binary Search can be used to locate a student's result quickly.

Binary Search is an efficient searching algorithm that works by repeatedly dividing a sorted dataset into two halves. It is especially useful when the number of student records is very large.

2. Problem Statement

The university needs a system that allows students or staff to search for examination results using a registration number. Searching every student record sequentially would be inefficient for a large database. Therefore, an algorithm is required that can provide fast and accurate results.

Binary Search is selected because it can reduce the search space by half at every step.

3. System Requirements

The system requires the following:

- **Sorted storage:** Records must be sorted according to registration number.
- **Unique registration number:** Every student should have a unique registration number.
- **Fast lookup:** The system should return results quickly.
- **Correctness:** The correct student record must be displayed.
- **Reliable data:** Student details and examination marks should be stored accurately.

- **Search failure handling:** The system should display an appropriate message when a registration number does not exist.

4. Binary Search Method

Binary Search follows the **divide-and-conquer** technique.

Suppose the sorted registration numbers are:

1001, 1005, 1010, 1015, 1020, 1025, 1030

If the user searches for registration number **1025**:

1. Find the middle element: 1015.
2. Compare 1025 with 1015.
3. Since $1025 > 1015$, ignore the left half.
4. Search the right half.
5. The next middle value is 1025.
6. The student record is found.

Thus, a large number of records can be eliminated after every comparison.

5. Algorithm

- 1: Start.
- 2: Store student records in ascending order of registration number.
- 3: Read the registration number to be searched.
- 4: Initialize:
 - $low = 0$
 - $high = n - 1$
- 5: Calculate the middle position:
 $mid = (low + high) // 2$
- 6: Compare the entered registration number with the middle record:
 - If $reg_no == records[mid]$, display the student's result.

- If `reg_no < records[mid]`, set `high = mid - 1`.
- If `reg_no > records[mid]`, set `low = mid + 1`.

7: Repeat Steps 5 and 6 while `low <= high`.

8: If the registration number is not found, display "Student Record Not Found".

9. Stop.

Python Code:

```
File Edit Format Run Options Window Help
# Digital Examination Result Search System
# Using Binary Search

students = [
    (1001, "Arun", 85, "Pass"),
    (1005, "Bala", 76, "Pass"),
    (1010, "Charan", 91, "Pass"),
    (1015, "Divya", 68, "Pass"),
    (1020, "Esha", 45, "Pass"),
    (1025, "Farhan", 88, "Pass"),
    (1030, "Gita", 32, "Fail")
]

key = int(input("Enter Registration Number: "))

low = 0
high = len(students) - 1
found = False

while low <= high:
    mid = (low + high) // 2

    if students[mid][0] == key:
        print("\nStudent Record Found")
        print("Registration Number:", students[mid][0])
        print("Student Name:", students[mid][1])
        print("Marks:", students[mid][2])
        print("Result:", students[mid][3])
        found = True
        break

    elif key < students[mid][0]:
        high = mid - 1

    else:
        low = mid + 1

if not found:
    print("Student Record Not Found")
```

Output:

```
===== RESTART: C:/Users/Tina
Enter Registration Number: 1025

Student Record Found
Registration Number: 1025
Student Name: Farhan
Marks: 88
Result: Pass
```

Divide and Conquer Principle:

Binary Search uses three main divide-and-conquer steps:

Divide:

The sorted list is divided into two halves by finding the middle element.

Conquer:

Only the half that may contain the required registration number is selected.

Repeat:

The same process is repeatedly applied to the selected half until the student record is found.

Therefore, instead of searching all records, Binary Search continuously reduces the search area.

Time Complexity Analysis

For n student records:

Case	Time Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$
Space Complexity	$O(1)$

Advantages:

- Very fast for large sorted datasets.
- Reduces the search space by half at every step. Requires only $O(\log n)$ time.
- Simple and efficient to implement. Provides quick access to examination results.
- Uses very little additional memory in the iterative implementation.

Conclusion

Binary Search is highly suitable for a Digital Examination Result Search System because student registration numbers can be maintained in sorted order. Using the divide-and-conquer approach, Binary Search repeatedly eliminates half of the records that cannot contain the required student. Its $O(\log n)$ time complexity makes it significantly faster than Linear Search for large numbers of student records. Therefore, Binary Search provides a fast, efficient, and reliable method for accessing university examination results

2.Robotics Control Transformation Engine Using Matrix Multiplication

Introduction

A robotics control system uses matrices to calculate motion transformations, rotation, translation, and coordinate conversion. Robotic arms repeatedly multiply transformation matrices to determine the position and orientation of their joints. Therefore, an efficient matrix multiplication method is important for real-time operation.

Requirements

The system should provide:

- High precision for accurate robot movement.
- Fast computation for real-time control.
- Support for different matrix sizes.
- Correct transformation and coordinate calculations.
- Efficient memory usage.

Divide-and-Conquer Approach

Divide-and-conquer matrix multiplication divides a large matrix into smaller submatrices and multiplies them separately.

This approach is useful for large matrices because smaller operations can be processed efficiently and potentially in parallel.

Python Code:

```
def multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]

    return C

A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

C = multiply(A, B)

print("Result:")
for row in C:
    print(row)
```

Output:

```
===== RESTART: C:/Users/Thanushma/T
Result:
[19, 22]
[43, 50]
```

Precision and Matrix Size

Robotics commonly uses 3×3 or 4×4 transformation matrices. Floating-point calculations may introduce small rounding errors, especially after repeated multiplication. Therefore, suitable numerical precision should be maintained.

For small matrices, standard multiplication is generally better because it has low overhead. For very large matrices, optimized divide-and-conquer methods such as Strassen's algorithm can be considered.

Time Complexity

For standard matrix multiplication: $O(n^3)$

For Strassen's algorithm: $O(n^{2.807})$

The space complexity is generally: $O(n^2)$

Justification

For robotic arms, standard optimized matrix multiplication is usually preferred because transformation matrices are small and calculations must be predictable and fast. Divide-and-conquer techniques become more beneficial when dealing with larger matrices or many simultaneous calculations.

Conclusion

Matrix multiplication is essential for robotic motion and coordinate transformations. Divide-and-conquer can improve the organization and efficiency of large matrix calculations. However, for common 3×3 and 4×4 robotics transformations, standard optimized multiplication provides the best balance of speed, precision, simplicity, and real-time performance.